

ProductService.java

```
package com.harisudhan.harisudhanmart.service;

import com.harisudhan.harisudhanmart.dao.ProductDAO;
import com.harisudhan.harisudhanmart.dto.ProductRequestDTO;
import com.harisudhan.harisudhanmart.exception.NotFoundException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.model.Product;
import com.harisudhan.harisudhanmart.util.ValidationUtil;
import java.sql.SQLException;
import java.util.List;

public class ProductService {

    private final ProductDAO productDAO;

    public ProductService(ProductDAO productDAO) {
        this.productDAO = productDAO;
    }

    public Product create(long sellerId, ProductRequestDTO req) throws ValidationException, SQLException {
        validate(req);
        Product p = new Product();
        p.setSellerId(sellerId);
        applyFields(p, req);
        return productDAO.create(p);
    }

    public Product update(long id, long sellerId, ProductRequestDTO req)
        throws ValidationException, NotFoundException, SQLException {
        validate(req);
        Product p = productDAO.findById(id)
            .orElseThrow(() -> new NotFoundException("Product not found"));
        if (!p.getSellerId().equals(sellerId)) {
            throw new ValidationException("FORBIDDEN", "You do not own this listing");
        }
        applyFields(p, req);
        productDAO.update(p);
        return p;
    }

    public void delete(long id, long sellerId) throws NotFoundException, SQLException {
        boolean removed = productDAO.delete(id, sellerId);
        if (!removed) {
            throw new NotFoundException("Product not found or not owned by this seller");
        }
    }

    public List<Product> search(String keyword, String category) throws SQLException {
        return productDAO.search(keyword, category);
    }

    public List<Product> findBySeller(long sellerId) throws SQLException {
        return productDAO.findBySeller(sellerId);
    }

    public Product findByIdOrThrow(long id) throws NotFoundException, SQLException {
        return productDAO.findById(id).orElseThrow(() -> new NotFoundException("Product not found"));
    }

    private void applyFields(Product p, ProductRequestDTO req) {
        p.setName(req.getName().trim());
        p.setDescription(req.getDescription());
        p.setPrice(req.getPrice());
    }
}
```

```
        p.setStockQty(req.getStockQty());
        p.setCategory(req.getCategory());
        p.setImageUrl(req.getImageUrl());
    }

    private void validate(ProductRequestDTO req) throws ValidationException {
        if (ValidationUtil.isBlank(req.getName())) {
            throw new ValidationException("name", "Product name is required");
        }
        if (!ValidationUtil.isPositive(req.getPrice())) {
            throw new ValidationException("price", "Price must be a positive amount");
        }
        if (req.getStockQty() < 0) {
            throw new ValidationException("stockQty", "Stock quantity cannot be negative");
        }
    }
}
```